TP4: Algorithmes gloutons

CPES ≪ Sciences des données, arts et cultures ≫

Antoine GAUQUIER
antoine.gauquier@ens.fr

Pierre ABOULKER
pierreaboulker@gmail.com

03 décembre 2024

Dans ce TP, nous nous intéressons au problème du sac à dos, qui est un problème d'*optimisation combinatoire*. C'est un problème dit *NP-complet*, c'est à dire dont la solution est très longue voire impossible à obtenir en un temps raisonnable. Ce TP est largement inspiré d'un sujet proposé par Cyril Charignon.

Introduction et solutions possibles

Un voleur pénètre dans la maison d'un riche informaticien. Les objets les plus intéressants sont une collier de perles, un candelabre en argent, le livre *Introduction to Algorithms* de Cormen et al. [CLRS22] dédicacé par les auteurs, et une fourrure de glomorphe à rayures. Comme il doit rester léger pour escalader la façade de l'immeuble, il n'a pris qu'un petit sac à dos pouvant **contenir au maximum 8kg**. Le tableau 1 donne le poids et la valeur (en centaines d'euros) des différents objets.

TABLE 1 – Tableau des poids et valeurs des différents objets du problème

Objet	Poids (kg)	Valeur (centaines d'€)
Collier	1	15
Candelabre	5	10
Cormen	3	9
Glomorphe	4	5

Le voleur s'interroge sur les objets qu'il doit mettre dans son sac à dos. On envisage deux stratégies :

1. *Le voleur est glouton* : Comme vu en cours, il faut définir un *critère de gloutonnerie*. Ici celui-ci consistera à **prendre en priorité les objets de plus grande valeur**.
2. *Le voleur a le temps* : On suppose que le voleur décide d'essayer toutes les combinaisons possibles et de calculer, pour chacune d'entre elles, la valeur totale. Il choisira ensuite la meilleure solution.

Question 1 Pour la solution numéro 1, quels objets le voleur prendrait-il en suivant le critère de gloutonnerie décrit ? Ce choix est-il optimal ? Si non, donner celui qui le serait.

Question 2 Pour la solution numéro 2, on considère maintenant qu'il faut 5 secondes au voleur pour analyser une combinaison (déterminer si elle est possible ou non, puis calculer combien elle rapporterait). Combien de temps faut-il au voleur pour se décider ?

Solution gloutonne en Python

On commence par formaliser le problème du sac à dos. On considère un sac à dos de capacité C (dans l'exemple, $C = 8$kg). On considère $n \in \mathbb{N}$ objets différents, dont on note $p_0, \dots, p_{n-1}$ les poids et $v_0, \dots, v_{n-1}$ les valeurs. On considère que ces poids et valeurs sont *des entiers strictement positifs* (on pourrait utiliser des poids flottants, mais ça rendrait l'algorithme plus complexe), et que le poids de chaque objet ne dépasse pas la capacité totale. Le but du problème est de retourner un sous-ensemble $X \subseteq [\![0, n-1]\!]$ tel que :

— $\sum_{k \in X} p_k \leqslant C$

— $\sum_{k \in X} v_k$ est maximale.

Le critère de gloutonnerie présenté en introduction est un premier choix cohérent possible, mais il est possible de faire mieux. En effet, on veut maximiser la valeur tout en minimisant le poids : on peut donc prendre en priorité les objets qui ont **meilleur rapport valeur/poids**. On notera ce rapport $\alpha_i = \frac{v_i}{p_i}$, $\forall i \in [\![0, n-1]\!]$.

Question 3 Écrire une fonction `rapport_valeur_poids` prenant en entrée deux tableaux `valeurs` et `poids`, qui contiennent tous deux respectivement $\{v_i\}_{i \in [\![0,n-1]\!]}$ et $\{p_i\}_{i \in [\![0,n-1]\!]}$. Cette fonction retournera un tableau `alpha`, contenant $\{\alpha_i\}_{i \in [\![0,n-1]\!]}$.

Question 4 On veut maintenant écrire une fonction qui retourne le meilleur objet à prendre parmi les objets restants. Écrire une fonction `meilleur_objet_restant`, prenant en paramètre le tableau `alpha` des rapports valeur/poids, et un tableau `traites` de taille n qui, pour chaque indice $i \in [\![0, n-1]\!]$, contient la valeur `True` si l'objet i a déjà été considéré (auquel cas, on l'excluera systématiquement), et `False` sinon. La fonction retournera la valeur i de l'objet pas encore traité ayant le meilleur rapport α. Si jamais tous les objets sont déjà traités, on retournera -1. **On ne modifiera pas à ce stade le tableau** `traites`.

Question 5 Écrire finalement une fonction `glouton` prenant en entrée le tableau `valeurs`, le tableau `poids` et la capacité `C`. La fonction devra retourner un tableau `resultat` indiquant les indices des objets sélectionnés, ainsi que la valeur que représente ceux-ci qu'on notera `valeur_totale`. *Rappel : pour retourner deux objets* `o1` *et* `o2` *en Python, il suffit d'écrire «* `return o1, o2` *». Pour appeler la fonction, on pourra utiliser «* `o1, o2 = fonction(...)` *».*

Question 6 Afin de visualiser plus facilement la sortie de votre algorithme, écrivez une fonction `afficher_sortie_glouton`, prenant en paramètre `resultat`, `valeur_totale` et un tableau `noms_objets` qui contiendra, pour chaque indice $i \in [\![0, n-1]\!]$, le nom de l'objet i. Elle ne retournera rien, mais affichera la liste des objets choisis et la valeur totale correspondante.

Analyse de la solution gloutonne

Analysons maintenant la solution écrite en Python.

Question 7 Tester la fonction `glouton` sur l'exemple donné en introduction, puis afficher sa sortie à l'aide de `afficher_sortie_glouton`. La solution retournée est-elle cette fois-ci optimale ?

Question 8 Proposer un exemple de listes `valeurs` et `poids` pour lequel la solution retournée n'est pas optimale.

Question 9 Quelle est la complexité asymptotique temporelle de l'algorithme associé à la fonction `glouton` ? Justifier.

Question 10 En exploitant la fonction `sorted(l)` prenant en entré une liste Python `l` et retournant cette même liste triée, proposer une nouvelle fonction `glouton_rapide` dont l'algorithme associé a une complexité temporelle asymptotique en $O(n \log\ n)$; la complexité asymptotique temporelle de la fonction `sorted(l)` étant elle-même en $O(n \log\ n)$.

Références

[CLRS22] Thomas H Cormen, Charles E Leiserson, Ronald L Rivest, and Clifford Stein. *Introduction to Algorithms*. MIT press, 2022.